LAPORAN PRAKTIKUM

STRUKTUR DATA

Queue

disusun Oleh:

MUHAMAD FAJRI AULIA

NIM 2511532009

DOSEN PENGAMPU : DR. WAHYUDI, S.T, M.T

FAKULTAS TEKNOLOGI INFORMASI

DEPARTEMEN INFORMATIKA

UNIVERSITAS ANDALAS

PADANG, 28 APRIL 2026

KATA PENGANTAR

Puji syukur kehadirat Allah SWT yang telah memberikan rahmat, taufik, dan hidayahNya sehingga penulis dapat menyelesaikan laporan praktikum mata kuliah Struktur Data dengan baik dan tepat waktu. Laporan ini disusun untuk memenuhi salah satu tugas praktikum pada pertemuan kelima dengan judul "**Queue**". Melalui penyusunan laporan ini, penulis berharap dapat memberikan gambaran mengenai Queue dengan java.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, penulis sangat mengharapkan kritik dan saran yang membangun demi perbaikan laporan maupun pemahaman di masa yang akan datang. Akhir kata, penulis mengucapkan terima kasih kepada dosen pengampu serta semua pihak yang telah memberikan bimbingan, arahan, dan bantuan hingga laporan ini dapat terselesaikan. Semoga laporan ini dapat bermanfaat bagi pembaca.

Padang, 28 April 2026

Penulis

DAFTAR ISI

LAPORAN PRAKTIKUM 1
KATA PENGANTAR i
DAFTAR ISI ii
DAFTAR GAMBAR iii
BAB I PENDAHULUAN 1
1.1 Latar Belakang 1
1.2 Tujuan Praktikum 1
1.3 Persyaratan Praktikum 1
1.4 Waktu dan tempat pelaksanaan 2
BAB II PEMBAHASAN 3
2.1 Pengertian Stack 3
2.2 Operasi dasar pada sebuah stack 3
2.3 Langkah Pengerjaan 3
2.3.1 Queue Array 3
2.3.2 Queue Array Driver 5
2.3.3 Queue Linkedlist 6
2.3.4 Reverse Data 7
2.3.5 Iterasi Queue 8
BAB III KESIMPULAN DAN SARAN 10
3.1 Kesimpulan 10
3.2 Saran 10
BAB IV LAPORAN TUGAS 11
4.1 Kode Program 11
4.2 Driver Program 12
4.3 Output yang dihasilkan 13
DAFTAR PUSTAKA 15

DAFTAR GAMBAR

Gambar 1 6
Gambar 2 7
Gambar 3 8
Gambar 4 9
Gambar 5 13
Gambar 6 14
Gambar 7 14

BAB I
PENDAHULUAN

1.1 Latar Belakang

Queue atau antrian merupakan sebuah struktur data linier yang mengikuti urutan tertentu Dimana operasi dilakukan. Biasanya urutannya FIFO (First In First Out) yang mana ini maksudnya bahwa elemen yang pertama kali dimasukkan maka akan keluar pertama dan menyatakan bahwa element yang dimasukkan pertama akan keluar pertama.

1.2 Tujuan Praktikum

- Memahami konsep dasar Queue di Java.
- Mengetahui cara membuat dan menggunakan Queue
- Melatih penggunaan method yang ada pada Queue.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut :

- Telah mengikuti perkuliahan teori Struktur data sebagai dasar pemahaman.
- Membawa perlengkapan yang diperlukan, antara lain laptop atau computer yang sudah terpasang Java Development Kit (JDK) dan Integrated Development Enviroment (IDE) yang direkomendasikan.
- Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
- Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
- Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman ini.

1.4 Waktu dan tempat pelaksanaan

Pelaksanakan praktikum Java mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanankan secara mandiri dengan perangkat masing masing, selama memenuhi syarat teknis yang ditetapkan.

BAB II
PEMBAHASAN

2.1 Pengertian Stack

Queue adalah struktur data yang menyusun elemen elemen data dalam urutan linier . Prinsip dasar dari queue adalah First In First Out (FIFO) yang mana setiap data yang pertama kali dimasukkan ke dalam antrean akan menjadi yang pertama pula yang dikeluarkan. Pada struktur data ini, urutan pertama (data yang akan dikeluarkan) disebut Front atau Head.

Sebaliknya, data pada urutan terakhir (data yang baru saja ditambahkan) disebut Back, Rear atau Tail. Proses untuk menambahkan data pada antrean disebut dengan Enqueue sedangkan proses untuk menghapus data dari antrean disebut dengan Dequeue..

2.2 Operasi dasar pada sebuah stack

Beberapa operdasi dasar dari sebuah stack adalah sebagai berikut :

- Enqueue (Sisipkan) : Menambahkan elemen ke bagian belakang queue.
- Dequeue (Hapus) : Menghapus dan mengembalikan elemen dari bagian depan queue.
- Peek : Return elemen di bagian depan queue tanpa menghapusnya.
- Empty (Kosong) : Memeriksa apakah queue kosong.
- Full : Memeriksa apakah queue sudah penuh.

2.3 Langkah Pengerjaan

Untuk lebih memahami kita akan mulai membahas program berikut.

2.3.1 Queue Array

Untuk kode programnya sebagai berikut.

```
package pekan4_2511532009;

public class QueueArray_2511532009 {
	int front_2009, rear_2009, size_2009;
	int capacity_2009;
	int array_2009[];

	public QueueArray_2511532009(int capacity_2009) {
		this.capacity_2009 = capacity_2009;
		front_2009 = this.size_2009 = 0;
		rear_2009 = capacity_2009 -1;
```

```java
            array_2009 = new int[this.capacity_2009];
        }

        boolean isFull_2511532009 (QueueArray_2511532009 queue_2009) {
            return (queue_2009.size_2009 == queue_2009.capacity_2009);
        }

        boolean isEmpty_2511532009 (QueueArray_2511532009 queue_2009) {
            return (queue_2009.size_2009 == 0);
        }

        void enqueue_2009 (int item_2009) {
            if (isFull_2511532009(this))
                return;
            this.rear_2009 = (this.rear_2009 + 1) % this.capacity_2009;
            this.array_2009[this.rear_2009] = item_2009;
            this.size_2009 = this.size_2009 + 1;
            System.out.println(item_2009 + " enqueued to queue");
        }

        int dequeue_2009() {
            if (isEmpty_2511532009(this))
                return Integer.MIN_VALUE;
            int item_2009 = this.array_2009[this.front_2009];
            this.front_2009 = (this.front_2009 + 1) % this.capacity_2009;
            this.size_2009 = this.size_2009 - 1;
            return item_2009;
        }
        int front_2009 () {
            if (isEmpty_2511532009(this))
                return Integer.MIN_VALUE;
            return this.array_2009[this.front_2009];
        }
        int rear_2009() {
            if (isEmpty_2511532009(this))
                return Integer.MIN_VALUE;
            return this.array_2009[this.rear_2009];
        }

        //mencetak elemen antrian
        void display () {
            int i_2009;
            if (front_2009 == rear_2009) {
                System.out.println("\nAntrian Kosong|n");
                return;
            }
            //kunjungi dari belakang dan cetak
            for (i_2009 = front_2009; i_2009 < rear_2009; i_2009++) {
                System.out.printf(" %d <-- ", array_2009[i_2009]);
            }
            return;
        }

}
```

Pada program ini kita mendeklarasikan beberapa variabel dengan tipe tertentu lalu kita membuat sebuah method Dimana tujuannya mengembalikan nilai true jika

ukuran array sudah sama dengan kapasitas. Selanjutnya kita membuat sebuah method untuk mengembalikan nilai true jika ukuran array sama dengan 0. Selanjutnya kita buat sebuah method yang mana menampilkan proses penambahan elemen bar ke dalam antrian lalu jika queue sudah penuh akan tidak melakukan apapun. Lalu kita buat sebuah method penilai jika sebiah array kosong yang mana jika kosong maka akan mengembalikan nilai sebagai eror. Selain itu kita membuah sebuah method untuk menampilkan isi antrian lalu kita buat perulangan yang akan menampikan nilai indeks. Untuk outputnya bisa dilihat dari bagian driver.

2.3.2 Queue Array Driver

Untuk program dari Queue Array driver ada sebagai berikut.

```
package pekan4_2511532009;

public class QueueArrayDriver_2511532009 {

	public static void main(String[] args) {
		QueueArray_2511532009 queue_2009 = new QueueArray_2511532009(1000);
		queue_2009.enqueue_2009(10);
		queue_2009.enqueue_2009(20);
		queue_2009.enqueue_2009(30);
		queue_2009.enqueue_2009(40);
		System.out.println("Item didepan "+ queue_2009.front_2009());
		System.out.println("Item paling belakang " +
queue_2009.rear_2009());
		System.out.println("Tampilakn Queue");
		queue_2009.display();
		System.out.println();
		System.out.println(queue_2009.dequeue_2009() + " dihapus dari
queue");
		System.out.println("Item didepan: " + queue_2009.front_2009());
		System.out.println("Item dibelakang: " + queue_2009.rear_2009());
		System.out.println("Tampilan queue setelah satu data dihapus");
		queue_2009.display();


	}

}
```

Disini kita menambahkan elemen kedalamm untuk dimasukkan ke antrian lalu kita memanggil item yang paling depan lalu mengembalikan nilai elemen paling belakgan , menampilkan isi antrian. Lalu kita juga memanggil method untu menghapus elemen menampilkan elemen paling depan lalu memanggil elemen

paling belakang dan isi antrian setelah adanya penghapusan dengan hasil output adalah sebagai berikut.

```
10 enqueued to queue
20 enqueued to queue
30 enqueued to queue
40 enqueued to queue
Item didepan 10
Item paling belakang 40
Tampilakn Queue
 10 <--  20 <--  30 <--
10 dihapus dari queue
Item didepan: 20
Item dibelakang: 40
Tampilan queue setelah satu data dihapus
 20 <--  30 <--
```

Gambar 1

2.3.3 Queue Linkedlist

Untuk program dari Queue Linkedlist adalah sebagai berikut.

```java
package pekan4_2511532009;
import java.util.*;
import java.util.Queue;
public class QueueLinkedList_2511532009 {

    public static void main(String[] args) {
        Queue<Integer> q_2009 = new LinkedList<> ();
        // tambah elemen {0, 1, 2, 3, 4, 5} ke antrian
        for (int i_2009 = 0; i_2009 < 6; i_2009++)
            q_2009.add(i_2009);
        // Menampilkan isi antrian.
        System.out.println("Elemen Antrian " + q_2009);
        // Untuk menghapus kepala antrian.
        int hapus_2009 = q_2009.remove();
        System.out.println("Hapus elemen = " + hapus_2009);
        System.out.println(q_2009);
        // Untuk melihat antrian terdepan
        int depan_2009 = q_2009.peek();
        System.out.println("Kepala Antrian = " + depan_2009);

        int banyak_2009 = q_2009.size();
```

```
        System.out.println("Size Antrian = " + banyak_2009);



    }

}
```

Pada program ini kita membuat sebuah queue dengan menggunakan linkedlist lalu menambah nilai ke dalam queue kemudian kita menampilkan isi dari queue lalu untuk menghapus kepala dari queue kita gunakan remove lalu untuk menampilkan data yang di hapus kita panggil variabel hapus lalu bis akita lakukan untuk menampilkan antrian terdepan dengan peekm lalu menampilkan ukuran atau size dari queue dengan output adalah sebagai berikut.

```
Elemen Antrian [0, 1, 2, 3, 4, 5]
Hapus elemen = 0
[1, 2, 3, 4, 5]
Kepala Antrian = 1
Size Antrian = 5
```

Gambar 2

2.3.4 Reverse Data

Untuk program dari Reverse Data adalah sebagai berikut.

```
package pekan4_2511532009;
import java.util.*;
public class ReverseData_2511532009 {

	public static void main(String[] args) {
		Queue<Integer> q_2009 = new LinkedList <Integer>();
		q_2009.add(1);
		q_2009.add(2);
		q_2009.add(3); // [1, 2, 3]
		System.out.println("Sebeleum reverse" + q_2009);
		Stack<Integer> s_2009 = new Stack<Integer>();
		while (!q_2009.isEmpty()) { // Q -> S
			s_2009.push(q_2009.remove());
	}
		while (!s_2009.isEmpty()) { // S -> Q
			q_2009.add(s_2009.pop());
		}
		System.out.println("Sesudah reverse = " + q_2009);// [3, 2, 1]
```

```
}
}
```

Pada proram ini kita membuat sebuah queue dengan menggunakan linkedlist kemudain kita menggunakan class add untuk menambahkan elemen kedalam antrian yang mana elemen tadi bisa kita tampilkan dengan cara memanggil queue tadi selanjutnya kita membuat objek stack yang mana kita buat perulangan Dimana akan terus berulang selama queue tidak kosong dan stack akan terisi dari elememn dari antrian yang sudah diremove. Kemudian kita buat sebuah perulangan lagi Dimana ini akan terulang selama stack belum kosong lalu kita bisa menampilkan hasil dari queue yang setelah dibalik menggunakan beberapa Langkah sebelumnnya. Dengan output sebagai berikut.

```
Sebeleum reverse[1, 2, 3]
Sesudah reverse = [3, 2, 1]
```

Gambar 3

2.3.5 Iterasi Queue

Pada program Iterasi Queue adalah proses Dengan program adalah sebagai berikut.

```java
package pekan4_2511532009;
import java.util.*;
public class IterasiQueue_2511532009 {

	public static void main(String[] args) {
		Queue<String> q_2009 = new LinkedList<> ();

		q_2009.add("Praktikum");
		q_2009.add("Struktur");
		q_2009.add("Data");
		q_2009.add("Dan");
		q_2009.add("Algoritma");
		Iterator<String> iterator_2009 = q_2009.iterator();
		while (iterator_2009.hasNext()) {
			System.out.print(iterator_2009.next() + " ");
		}
```

```
        }
}
```

Pada program ini kita membuat sebuah queue dan menambahkan elemen kedalam queue lalu kita gunakan method iterator untuk menghasilkan objek yang siap menelusuri koleksi yang mana kita buat perulangan Dimana ini akan terus terjadi selama iterator bergantung pada hasnext bernilai benarjika masih maka elemen berikutnya belum bisa diakses dan jika salah maka iterator sudah mencapai akhir dengan output yang dihasilkan adalah sebagai berikut.

```
Praktikum Struktur Data Dan Algoritma
```

Gambar 4

BAB III
KESIMPULAN DAN SARAN

3.1 Kesimpulan

Struktur Data Queue (Antrian) adalah konsep dasar dalam ilmu komputer yang digunakan untuk menyimpan dan mengelola data dalam urutan tertentu. Antrian mengikuti prinsip " First in, First out " (FIFO), di mana elemen pertama yang ditambahkan ke dalam queue adalah elemen pertama yang dihapus. Antrian biasanya digunakan dalam berbagai algoritma dan aplikasi karena kesederhanaan dan efisiensinya dalam mengelola aliran data Antrian beroperasi seperti garis di mana elemen ditambahkan pada satu ujung (rear) dan dihapus dari ujung lainnya (front). Bayangkan sebuah antrian seperti orang-orang yang mengantre di supermarket. Orang pertama yang mengantri adalah orang pertama yang dapat membayar dan meninggalkan supermarket

3.2 Saran

- Ulangi dan terus Latihan agar makin memahami konsep dari Queue.
- Sertakan Langkah kerja program secara ruhut dan sistematis.

BAB IV LAPORAN TUGAS

4.1 Kode Program

```java
package pekan4_2511532009;

public class AntrianLoket_2511532009 {
    private String[] data_2009;
    private int front_2009;
    private int rear_2009;
    private int kapasitas_2009;
    private int jumlah_2009;

    // Konstruktor
    public AntrianLoket_2511532009(int kapasitas_2009) {
        this.kapasitas_2009 = kapasitas_2009;
        this.data_2009 = new String[kapasitas_2009];
        this.front_2009 = 0;
        this.rear_2009 = -1;
        this.jumlah_2009 = 0;
    }

    // Mengecek apakah kosong
    public boolean kosong_2009() {
        return jumlah_2009 == 0;
    }

    // Mengecek apakah penuh
    public boolean penuh_2009() {
        return jumlah_2009 == kapasitas_2009;
    }

    // Tambah antrian
    public void tambah_2009(String nama_2009) {
        if (penuh_2009()) {
            System.out.println("Antrian penuh, tidak bisa menambah!");
            return;
        }
        rear_2009 = (rear_2009 + 1) % kapasitas_2009;
        data_2009[rear_2009] = nama_2009;
        jumlah_2009++;
        System.out.println(nama_2009 + " masuk ke antrian.");
    }

    // Hapus antrian
    public void hapus_2009() {
        if (kosong_2009()) {
            System.out.println("Antrian kosong, tidak ada yang dihapus!");
            return;
        }
        String keluar_2009 = data_2009[front_2009];
        front_2009 = (front_2009 + 1) % kapasitas_2009;
        jumlah_2009--;
        System.out.println(keluar_2009 + " telah dilayani.");
    }

    // Tampilkan isi antrian
    public void tampil_2009() {
```

```java
        if (kosong_2009()) {
            System.out.println("Isi antrian: Kosong");
            return;
        }
        System.out.println("Isi antrian:");
        for (int i = 0; i < jumlah_2009; i++) {
            int idx = (front_2009 + i) % kapasitas_2009;
            System.out.println((i + 1) + ". " + data_2009[idx]);
        }
    }

    // Membalik isi antrian
    public void balik_2009() {
        if (kosong_2009()) {
            System.out.println("Antrian kosong, tidak bisa dibalik!");
            return;
        }
        String[] temp = new String[jumlah_2009];
        for (int i = 0; i < jumlah_2009; i++) {
            int idx = (front_2009 + i) % kapasitas_2009;
            temp[i] = data_2009[idx];
        }
        for (int i = 0; i < jumlah_2009; i++) {
            data_2009[i] = temp[jumlah_2009 - 1 - i];
        }
        front_2009 = 0;
        rear_2009 = jumlah_2009 - 1;
        System.out.println("Antrian berhasil dibalik.");
    }
}
```

4.2 Driver Program

```java
package pekan4_2511532009;

import java.util.Scanner;

public class AntrianLoketDriver_2511532009 {
    public static void main(String[] args) {
        Scanner input_2009 = new Scanner(System.in);
        AntrianLoket_2511532009 antrian_2009 = new AntrianLoket_2511532009(10);

        int pilihan;
        do {
            System.out.println("\n=== MENU ANTRIAN LOKET ===");
            System.out.println("1. Tambah Antrian");
            System.out.println("2. Hapus Antrian");
            System.out.println("3. Lihat Antrian");
            System.out.println("4. Balikkan Antrian");
            System.out.println("5. Keluar");
            System.out.print("Pilihan: ");
            pilihan = input_2009.nextInt();
            input_2009.nextLine();

            switch (pilihan) {
```

```java
                case 1:
                    System.out.print("Masukkan nama pelanggan: ");
                    String nama = input_2009.nextLine();
                    antrian_2009.tambah_2009(nama);
                    break;
                case 2:
                    antrian_2009.hapus_2009();
                    break;
                case 3:
                    antrian_2009.tampil_2009();
                    break;
                case 4:
                    antrian_2009.balik_2009();
                    antrian_2009.tampil_2009();
                    break;
                case 5:
                    System.out.println("Program selesai.");
                    break;
                default:
                    System.out.println("Pilihan tidak valid.");
            }
        } while (pilihan != 5);

        input_2009.close();
    }
}
```

4.3 Output yang dihasilkan

```
=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 1
Masukkan nama pelanggan: Fajri
Fajri masuk ke antrian.

=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 1
Masukkan nama pelanggan: Aull
Aull masuk ke antrian.

=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 3
Isi antrian:
1. Fajri
2. Aull
```

Gambar 5

```
=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 4
Antrian berhasil dibalik.
Isi antrian:
1. Aull
2. Fajri

=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 2
Aull telah dilayani.

=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 3
Isi antrian:
1. Fajri
```

Gambar 6

```
=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 4
Antrian berhasil dibalik.
Isi antrian:
1. Fajri

=== MENU ANTRIAN LOKET ===
1. Tambah Antrian
2. Hapus Antrian
3. Lihat Antrian
4. Balikkan Antrian
5. Keluar
Pilihan: 5
Program selesai.
```

Gambar 7

DAFTAR PUSTAKA

[1] Modul Praktikum Struktur Data Pekan 4

[2] Data Structures and Algorithms in Java